
CS 289A Graduate Project

Overview

This assignment asks you to pick a problem that interests you, do serious technical work on it, and write up what you found.

Your project might build on a paper you want to extend, a dataset you want to investigate, or a technique from lecture applied to a new domain. Think of the deliverable as a portfolio piece that you could present in an interview or hand to a collaborator to show what you're capable of.

Deliverables and Timeline

Deliverable	Due Date	Format
Project proposal	Friday, April 17, 11:59 PM PST	PDF, ~1 page
Final report + code	Friday, May 15, 11:59 PM PST	PDF + GitHub/zip

Teams of 2–3 students. The final report should be roughly 6–10 pages in a single-column format (ICML style or similar). The project is worth 100 points total.

You are free to use any resources to learn: papers, blog posts, reference implementations, LLMs. Cite any code you adapt. The standard is that you should be able to explain every part of your submission.

Proposal

The proposal is graded for completion. Its purpose is to let us flag issues with feasibility or scope before you invest weeks of work. If your proposal looks good, you may not hear back; if it needs adjustment, we will reach out.

Your proposal should cover:

- What problem you plan to work on and why it interests you.
- A brief description of the approach you intend to take.
- What you expect your contribution to be.
- Team member names and a rough division of responsibilities.
- A timeline for the remaining weeks through May 15.

The proposal is a checkpoint, not a contract. You are free to change direction afterward.

Final Report

Your report and codebase will be evaluated against the components below. Together they define what a complete project looks like.

1. Problem Definition

10 points

Frame your project around a clear technical question or goal. What are you investigating, and why is it worth investigating? A reader should finish this section with an unambiguous picture of what you set out to do.

2. Connection to Course Content

5 points

Identify the specific concepts, techniques, or mathematical tools from lecture that your work builds on. We want to see that this project is rooted in something you learned in this course, and that you can articulate the connection clearly.

3. Prior Work

10 points

Survey what exists. Methods, systems, papers, open-source implementations: whatever is relevant to the problem you chose. Show that you know what approaches have been tried, and that you can speak to where they succeed and where they fall short. This context is what makes your own contribution legible.

4. Approach

15 points

Describe what you did and how you did it, with enough technical precision that a knowledgeable reader could follow your reasoning end to end. Implementation choices, experimental setup, architecture decisions. If you're building on someone else's method, explain the core ideas in your own words; show that you've internalized it. Reimplementations of existing methods would belong here.

5. Your Contribution

25 points

This is the core of the project. What did you do that required your own thinking, your own judgment? Some examples of what a contribution can look like:

- An ablation testing a hypothesis no one has explored
- A controlled comparison between competing approaches under matched conditions
- A known method applied to a new problem or dataset
- An extension or modification of an existing technique, evaluated rigorously

A common thread across these examples is that they build on existing works at a modest scale.

6. Results and Discussion

20 points

Present your findings with clear figures and quantitative metrics. What worked, what didn't, and why? If something surprised you, say so and reason about it. If an experiment failed, analyze the failure; a negative result, honestly examined, reveals more about the depth of your engagement with the problem than a tidy positive result without deeper examination.

7. References

5 points

Cite the papers, codebases, blog posts, and any other sources your work draws on. Any standard citation format is fine.

8. Code

10 points

Submit a GitHub repository or zip archive with a README explaining how to run your key experiments. Clarity and correctness matter. The core logic should be written by your team, and you should be able to walk through any part of it and explain what it does.

Example

Here are two examples of what a strong project might look like. Your project does not need to resemble either one.

A team of two trains a ResNet-18 on CIFAR-10, achieving the expected 93% test accuracy. They freeze the trained network and fit a logistic regression classifier on the features from its penultimate layer. This linear probe reaches 91%, nearly matching the full network.

For their contribution, they evaluate both on corrupted test images (noise, blur, and contrast changes at five severity levels). The linear probe holds up better under every corruption type, losing roughly half as much accuracy under heavy noise. They hypothesize that the network's final layer overfits to patterns specific to clean data. To test this, they repeat the experiment with three architectures of increasing depth and find that the robustness gap grows with depth. The report analyzes this through the bias-variance tradeoff.

A team of three builds a classifier to predict the differentiation state of stem cells from microscopy images. They use a public dataset of cell colony images labeled across four stages of differentiation. A convolutional neural network trained on these images reaches 89% accuracy. They extract features from the network's penultimate layer and visualize them with PCA, confirming that the four stages separate into distinct clusters in feature space.

For their contribution, they investigate whether differentiation outcome can be predicted from images taken 24 hours before conventional markers become visible. They train a logistic regression classifier on early-timepoint features and find that it identifies colonies likely to fail differentiation with 78% precision, compared to 55% from a baseline using only colony area and shape. The report analyzes which learned features drive the early predictions and connects the classification boundary to the geometry of the PCA projections discussed in lecture.

Practical Advice

- Start small. Validate on toy data or a small model before scaling up; the most common failure mode is spending weeks debugging at full scale when the bug would have been obvious on a 2D example.
- Log everything. Weights & Biases, TensorBoard, even a well-organized folder of CSVs. You will re-run fewer experiments if you can actually find your old results.

- Use existing infrastructure wisely. Libraries like PyTorch, HuggingFace, TRL, and diffusers are fair game for data loading, model architectures, and training utilities; write the core logic yourself so you actually learn it.
- Come to office hours. We would rather help you course-correct in late April than read a frustrated report on May 15.